Prompt Log

Prayog: Backend Framework, Authentication & Authorisation, and review of the GPU Scheduling integration

Name	C Disha
Roll number	220302
Project	Prayog (team project; other modules: GPU scheduling, MLflow integration, AI)
My part	The backend framework the other modules are built on; authentication and authorisation; reviewing how the GPU scheduling module fits into that framework (structure, schema links, database conflicts) and merging its PRs
Period	8 Oct to 15 Oct 2026
Tool	Claude, used as a coding assistant

How to read this log

Each entry is a prompt I wrote while building my part of the backend, lightly cleaned for typos. Under each one I've written the logic or requirement I was giving the AI, the design decision that came out of that step, the alternatives I considered, and how I verified the result. The entries are grouped by phase, in the order they happened. My general approach was to describe the rule in plain words first, paste the real files or errors, take small steps, and push back whenever the answer didn't match how our system should behave.

Phase F covers my review of the GPU scheduling module. Those comments were given to the GPU owner (Adithya) rather than typed to my own assistant, and his AI engineering log records them with timestamps. I quote them as his log does and cite his entry IDs, so they can be checked against his document.

Phase	What it covers	Entries
A. Authorisation logic	Two-party approval, the state machine, locking, revoke	A1–A3
B. Framework structure	Wiring routes, schemas against logic, models, full review and layering	B1–B4
C. Database and infrastructure	SQLite vs PostgreSQL, Docker, Alembic, schema drift	C1–C5
D. Authentication and password reset	Professor onboarding, four iterations of reset, OTP design, role at login	D1–D6
E. Running and verifying	Uvicorn, Swagger testing, OTP end to end, git	E1–E3
F. The GPU module on my framework	My database setup underneath it, layout rules, foreign-key design against the revoke flow, conflicts, syncing, merging	F0–F5

Phase A: Authorisation logic

A1 Stating my two-party approval rule

8 Oct, 17:08

> **MY PROMPT**
>
> *see now look at this properly this is my logic: once the professor accepts/approves in his portal we update the database for the user_id he is approving. If status is rejected we make the end status rejected, because if either admin or professor rejects it's rejected. If it's approved we check the other; if both approved then final status approved, then move it to the right role table. This is one part, the other part is putting the request.*

Logic I gave	Two independent decisions (professor, admin) combine into one final status: any reject means rejected, both approve means approved, and approval moves the user into the Student or Researcher table.
What I asked for	A review of *my* rule and code, not a new design.
Issues it found	(1) A rejected request could later flip to approved and create a duplicate student, because the duplicate guard only checked for APPROVED. (2) Approving, then rejecting, left the role row in place. (3) `status` was never validated. (4) If professor and admin approved at the same moment, both could read the other as PENDING, so the request would never be finalised.
Next step	Its fix assumed every decision is final, which isn't how our portal works, so I corrected the rule in A2.

A2 Correcting it: rejection is reversible, approval locks

8 Oct, 17:09

> **MY PROMPT**
>
> *no, in the portal if accepted once it's done, but if you reject, later the professor has the option to change his decision / admin too. But once accepted we lock it, then only admin can remove, which is a different act. Based on this change my logic and give me the proper code.*

Logic I gave	Rejected is a *recoverable* state; approved-by-both is *terminal* for this action; removing access is a separate, admin-only action.
Design decision	Don't store `final_status` as an independent value. Recompute it from the two decisions after every change, so it can never disagree with them.
Alternatives considered	(a) Every decision final: too strict, since rejections are often mistakes. (b) Allow "reject" after approval: mixes two meanings and leaves orphaned role rows. Revoke as its own action was cleaner.
Concurrency	Lock the request row (`SELECT ... FOR UPDATE`) during a decision, so concurrent approvals run one after the other. UNIQUE constraints on `user_id` and `roll_number` in the role tables act as a second safety net.

The resulting state machine

```
final = REJECTED  if professor == REJECTED or admin == REJECTED
final = APPROVED  if professor == APPROVED and admin == APPROVED
                  -> create Student/Researcher row, request LOCKED
final = PENDING   otherwise

PENDING  <-> REJECTED      (either party may change their decision)
PENDING   -> APPROVED      (both approved; further decisions return 409)
APPROVED  -> REVOKED       (admin only; role row deleted; user may apply again)
```

A3 Getting the acting user from the JWT

8 Oct, 17:11

> **MY PROMPT**
>
> *current user getting that: [pasted my get_current_user dependency] ... is from jwt*

Logic I gave	Every authorisation check depends on knowing who is acting. That comes from my JWT dependency.
Issues it found	Install PyJWT, not the similarly named `jwt` package. PyJWT 2.10+ requires `sub` to be a string. A tampered `sub` like "abc" would cause a 500 instead of a 401.
Design decision	The role is *always re-read from the database*, never trusted from the token. So a role change takes effect immediately, and a forged role claim does nothing.
Outcome	One dependency (`get_current_user`) that every module now uses, plus `require_roles(...)` as a one-line route guard.

Phase B: Framework structure

B1 Wiring the routes into main.py

8 Oct, 17:14

MY PROMPT

[pasted main.py] this is my main.py, what and all should I add to this

Why this prompt	I pasted the real file so the answer would fit my code, not a generic template.
Issue it found	My `main.py` used `current_user.user_id` while the logic used `current_user.id`. Only one exists as the primary key.
Design decision	Two ids on purpose: `id` (integer) is the internal key for foreign keys and the JWT; `user_id` (the part of the email before @) is the public id.
Structure decision	Keep `main.py` for app setup only. Each feature gets its own router, included with one line. That's what lets other modules plug in without touching shared files.

B2 Checking the schemas against the logic

8 Oct, 17:15

MY PROMPT

[pasted schemas/access_request.py and schemas/auth.py] schema 1, schema 2

Why this prompt	Mismatches hide *between* files, so I gave it both schemas next to the logic it had already seen.
Issues it found	(1) The role enum was lowercase but every check compared uppercase, so no professor or admin could ever pass. (2) Anyone could register as ADMIN and approve their own request. (3) A leftover `professor_name` field would crash the submit route through `**body.model_dump()`.
Design decisions	All enum values are uppercase. Self-registration is limited to STUDENT and RESEARCHER; professors are created by an admin; the first admin comes from a terminal script. That closes the privilege-escalation path completely.

B3 Models, and a misread requirement

8 Oct, 17:19 to 17:20

> **MY PROMPT**
>
> *change models.py accordingly → no researcher has roll number → no I need roll_number for researchers too, in ours it's there*

What went wrong	I meant "no, a researcher *does* have a roll number". The AI read it as "researchers have no roll number" and removed the column.
Final requirement	Roll number is required and unique for both roles; only students have an end date.
Constraints added	Database-level `CHECK`s, so invalid data can't be saved even through a bug: valid role and status values, students need `end_date > start_date`, researchers need `end_date IS NULL`.
Lesson	Short negative phrasing is ambiguous. From then on I wrote requirements as complete positive statements.

B4 Full review before the team built on it

8 Oct, 17:23

> **MY PROMPT**
>
> *see this my current status, make everything proper, give me codes where and what I have to change, and tell me in detail why I need to change, and fix all the errors where the user registration, the access request, then storing them, everything is done, because after this we have lots of work*

Why this prompt	Other modules (GPU scheduling, MLflow, AI) were about to build on this. I uploaded every file and asked for the *why* behind each change, so I could defend the design to the team.
Crashes found before I hit them	`role.value` on a plain string broke registration; approving a researcher failed because the column was missing; comparing a timezone-aware date with a naive one crashed; and no route existed to create professors or admins at all.
Layering decision	`routers/` (HTTP only) → `schemas/` (validation) → `logic/` (rules) → `models.py` (data). Logic raises typed errors (400/403/404/409), and one handler in `main.py` converts them, so routes need no try/except.
Atomicity decision	Creating a professor writes a User and a Professor row. `flush()` gets the new id without committing, then one `commit()` saves both. Either both exist or neither does.

Phase C: Database and infrastructure

C1 SQLite vs PostgreSQL

8 Oct, 17:41

> **MY PROMPT**
>
> *[pasted my old database.py] my old database.py was this, should I change it to the new one*

Context	To test quickly, the AI had written a `database.py` that defaulted to SQLite. My project was already on PostgreSQL.
Design decision	Stay on PostgreSQL. SQLite ignores `SELECT ... FOR UPDATE`, so the approval race protection from A2 would silently do nothing. Postgres also matches what the team deploys.
Changes to my file	(1) Build the URL with `URL.create`, so a password containing @ : / # can't break it. (2) Fail clearly when a variable is missing, instead of connecting to host "None". (3) Remove the `SELECT 1` that ran on every import and move it into `/health`. (4) `pool_pre_ping` so the app recovers when Postgres restarts.
Health check logic	`/health` returns 200 only if the database answers, otherwise 503, so "API up but database down" is visible.

C2 PostgreSQL runs in Docker

8 Oct, 17:56 / 11 Oct, 18:01

> **MY PROMPT**
>
> *postgre sql is in docker → [later] CONTAINER ID 3b26207e674c IMAGE postgres:16 PORTS 0.0.0.0:5432->5432/tcp NAMES prayog-postgres*

Why this prompt	Docker networking changes what the host should be, so I told it how my database runs and later pasted my `docker ps` output.
Logic	The backend runs on my machine and Postgres in a container with port 5432 published, so `POSTGRES_HOST=localhost`. If the backend also ran inside docker-compose, the host would be the service name, because inside a container "localhost" means the container itself.
Data safety	`docker stop` keeps the data. `docker compose down -v` deletes the volume, and with it the database.
Lesson	Its first commands used `<postgres-container-name>` placeholders, which I typed literally. Once I pasted my real container name, the commands were exact.

C3 Alembic as the only owner of the schema

8 Oct, 17:55 to 17:57

> **MY PROMPT**
>
> *[pasted my alembic env.py] this is my env.py currently, what should I change → u don't need this kya [the duplicate URL.create block]*

Problem found	Both `main.py` (`create_all` on startup) and Alembic created tables. On a fresh database, the app creates the tables first, and then `alembic upgrade` fails with `DuplicateTable`. It reproduced this on a real Postgres.
Design decision	Only Alembic creates or alters tables. I removed `create_all` everywhere.
Single source of truth	`env.py` imports `DATABASE_URL` from `database.py` instead of building the URL again, so the app and the migrations can never point at different databases. My question "don't need this?" confirmed I could delete my duplicate block.
Detail	`compare_server_default=True`, otherwise autogenerate ignores changes to column defaults.

C4 Timestamps in UTC

8 Oct, 17:46

> **MY PROMPT**
>
> *this is my final struct, I have alembic setup too which I updated, so please check once if anything is broken or something*

Why this prompt	Before handing the base to the team I asked for one more check of everything, now on Postgres. That run surfaced this.
Problem	Postgres `now()` uses the server's timezone. In IST, `created_at` was stored 5½ hours ahead of the UTC request dates, which would break any later "has access expired" comparison.
Decision	Every timestamp defaults to `now() AT TIME ZONE 'utc'`, and incoming dates with a timezone are converted to UTC in the schema. One time base everywhere.
Verified	After migrating: `created_at` matched UTC now, not IST.

C5 Debugging schema drift in my Docker database

11 Oct, 17:59 to 18:02

MY PROMPT

[pasted traceback] psycopg2.errors.NotNullViolation: null value in column "created_at" of relation "users" violates not-null constraint → [pasted output of \d users]

Why this prompt	The traceback gave the symptom; the `\d users` output proved the cause.
Diagnosis	The model now leaves `created_at` to the database (`server_default`), but my table was from the old models, where the default lived in Python, so the column had no database default. The missing `ck_users_role` constraint confirmed the whole table was out of date.
Decision	Test data only, so: `DROP SCHEMA public CASCADE`, delete the old migrations, generate one clean initial migration, and read it line by line before applying.
Verified	The generated file had all 5 tables, the UTC defaults and every CHECK constraint; `alembic check` reported no differences.

Phase D: Authentication and password reset

The reset design went through four iterations. I've kept all of them, because the back-and-forth is where most of the design thinking happened.

D1 How a new professor gets an account

9 Oct, 16:43

> **MY PROMPT**
>
> *I have a doubt: when a professor is new, should he sign up or just log in directly, because in the user table will it be atomically added?*

Question behind it	Professors can't self-register (B2), so how do they get in, and is the account creation safe?
Answer	The admin creates them. `create_professor` writes the User and the Professor profile in one transaction, so a professor can't exist without a profile, or the other way round.
Gap it raised	At that point the admin chose the password, so the admin knew it, and there was no way for the professor to change it.

D2 Iteration 1: adding reset

9 Oct, 16:43

> **MY PROMPT**
>
> *then we need to give reset password too*

What it built	Change-password for logged-in users, forgot-password with an emailed one-time *link*, and an admin reset that sets a temporary password.
Security pieces I kept	`token_version`: a counter on each user, copied into every JWT. Every reset increments it, so **all old sessions die immediately**. JWTs are stateless, so this is the only server-side way to log someone out everywhere.
My assessment	It worked, but it was three separate mechanisms for one need.

D3 Iteration 2: first-login behaviour

9 Oct, 16:48

> **MY PROMPT**
>
> *set new password/reset password for professor logging in first time it should automatically show that but otherwise when clicked*

Logic I gave	First login on a temporary password should force the change; afterwards, changing the password is optional and done from a button.
What it built	A `must_change_password` flag, and a backend check returning `403 PASSWORD_CHANGE_REQUIRED` on every other route until the password is changed, so the rule holds even if the frontend forgets.
My assessment	Correct, but it added a flag, a special dependency and frontend routing, all just to handle a password the admin shouldn't know in the first place.

D4 Iteration 3: remove the temporary password

11 Oct, 19:41

> **MY PROMPT**
>
> *I'll change something: we'll just give the reset password option. We'll inform the professor when newly added that he must reset the password, as it is randomly generated, so that sorts the problem.*

Key insight	If the starting password is random and *nobody* knows it, there's nothing to force-change. The professor simply uses forgot-password like any other user.
Effect	Deleted change-password, the admin reset, the flag, the 403 block and the admin users routes. The admin never handles a password.
Design principle	Remove the cause instead of guarding against it.

D5 Iteration 4: OTP instead of a link

11 Oct, 19:44

> **MY PROMPT**
>
> *no no listen, just reset/forgot password, same method: email id with which you have registered, new password, then an OTP will be sent to the email id, type it in, password changes. That's all, give this code.*

Flow I specified	Step 1: email + new password → OTP emailed (nothing changes yet). Step 2: email + OTP → the stored new password takes effect.
Why OTP over a link	One screen with two fields instead of a separate reset page that has to read a token from a URL, and the user never leaves the app.
Storage decision	Step 1 stores the *Argon2 hash* of the new password plus an *HMAC* of the OTP. Neither is ever in plain text, and step 2 just copies the hash across.
Verified	The AI's automated run against Postgres covered every rule in the table below; then I ran the flow myself (E2).

OTP rules and the reasoning behind each

Rule	Value	Why
Length / source	6 digits, `secrets.randbelow`	Cryptographically random, not `random`.
Lifetime	10 minutes	Long enough to check email, short enough to limit guessing.
Wrong attempts	5, then cancelled	Guessing chance per OTP ≈ $5 / 10^6$ = 0.0005%.
Uses	Once; a new OTP cancels older ones	Only the newest code in the inbox works.
Resend limit	1 per minute per user	Stops the endpoint being used to spam someone's inbox.
Unknown email	Same 200 reply, nothing sent	The form can't be used to find out who has an account.
During resend limit	Same 200 reply, nothing sent	The AI first planned a "please wait" error. I agreed to drop it because only real accounts would ever see it, which leaks existence.

Rule	Value	Why
Hashing	HMAC-SHA256 with server secret + user id	A leaked database alone can't be brute-forced; the same code for two users hashes differently.
Comparison	`hmac.compare_digest`	Constant time, so response timing doesn't leak digits.
After success	`token_version += 1`	Every old session is logged out.

D6 Role selection at login

9 Oct, 16:49

> **MY PROMPT**
>
> *while logging in also role must be selected*

Logic	The login screen asks for a role, so the backend checks it: email + password + role must all match.
Design decision	A wrong role returns exactly the same 401 as a wrong password ("Invalid email, password or role"). Otherwise, trying each role with a known password would reveal which role an account has.
Verified	Correct password + wrong role gave 401; lowercase role accepted; missing role gave 422.

Phase E: Running and verifying

E1 Running the server

11 Oct, 20:05 to 20:09

> **MY PROMPT**
>
> *now lets run unicorn server → [and before that] okay give me alembic prompts to update → I want to see if alembic is updated properly → no no like I want to see after alembic revision --autogenerate -m "otp reset" has the new changes been actually taken*

Order I followed	`docker start prayog-postgres` → `alembic revision --autogenerate` → read the generated file → `alembic upgrade head` → `alembic check` → `uvicorn main:app --reload`.
Why I insisted on reading the migration	Autogenerate can silently miss changes or produce an empty file. I printed it with `cat "$(ls -t alembic/versions/*.py | head -1)"` and confirmed it contained exactly: create `password_reset_otps`, its index, and add `users.token_version` with default 0 (safe for existing rows). It contained nothing else, and `down_revision` pointed at my initial migration.
Checks after start	`/health` → `{"status":"ok","database":"ok"}`; `/docs` listed `forgot-password` and `verify-otp`.
Also confirmed	`main.py` needed no edits: `include_router(auth_router)` already exposed the new routes. Pasting router code into `main.py` would have defined them twice.

E2 Testing the OTP reset end to end

11 Oct, 20:12 to 20:14

> **MY PROMPT**
>
> *[screenshot: 422 string_too_short on new_password "dummy"] OTP didn't come when I was trying → [pasted the DEV EMAIL block with OTP 964227] I got this, now what should I do → yes worked*

First attempt	422: my test password "dummy" was 5 characters, below the 8-character minimum. That was the validation working, not a bug.
Second attempt	With a valid password, the OTP was printed in the uvicorn terminal (dev mode: no SMTP configured).
Then	`POST /auth/verify-otp` with the OTP → 200; login with the new password and role → token; the old password → 401; reusing the same OTP → 400.
What this proved	The whole chain works on my real setup: Docker Postgres, the Alembic schema, hashing, OTP verification, `token_version`, and role-checked login.

E3 Committing

11 Oct, 20:18

> **MY PROMPT**
>
> *git add. → git: 'add.' is not a git command*

Fix	`git add .` needs a space.
Before committing	Checked `git status` and made sure `.gitignore` excluded `.env` (database password, JWT secret), `.venv/` and `__pycache__/`. Committed the migration files together with the model changes, so teammates get the same schema with `alembic upgrade head`.

Phase F: The GPU module on my framework, and my review of it

Once the framework was in place, the GPU scheduling module (owner: Adithya) was the first large module built on it. My job here was to make sure it plugged into the framework instead of growing a second one: same app, same `models.py` and migration history, same users table, same auth. The quotes below are recorded in his AI engineering log; I cite his entry numbers.

F0 The database the GPU module was built on

8 to 11 Oct (before the GPU implementation began on 13 Oct)

> **WHAT I SET UP**
>
> *[database setup: PostgreSQL in Docker, database.py, models.py, Alembic env.py and the initial migration, all set up by me before the GPU work started (Phase C)]*

What I owned	The PostgreSQL 16 instance in Docker, the engine and session factory in `database.py`, the single `Base` in `models.py`, the Alembic `env.py` and the initial migration that creates `users`, `professors`, `access_requests`, `students` and `researchers`.
Why the GPU work needed it	His claim transaction uses `SELECT ... FOR UPDATE SKIP LOCKED` and partial unique indexes. Both exist only in PostgreSQL, which is why I kept the project on Postgres and not SQLite (C1). His log says the same when it rules SQLite out (his B1).
Migration chain	One migration history for the whole backend: my initial schema first, the GPU migrations after it, so `alembic upgrade head` builds auth and GPU together. His log reports a single head and zero drift (his C3 and verification ledger).
Conventions set for every module	UTC timestamps (`now() AT TIME ZONE 'utc'`), `CHECK` constraints for fixed value sets, `users.id` as the identity every table points to, and Alembic as the only way to change tables. I wrote these into the framework README's "building other modules" section.
Why one session setup matters	With one engine and a session per request, a row lock taken inside a transaction (like his claim) lasts exactly as long as that request's transaction and is released at commit. That's why I insisted on one app and one database layer (F1) rather than a second backend.

F1 Layout rules for the GPU code

13 Oct, 11:57 (Adithya's log C7)

> **MY REVIEW COMMENT (relayed to the GPU owner)**
>
> *Disha: In backend we have main and models. U need to update their directly. Dont create a separate file. And for the logic and schema and routers part, create a sub folder there and push them there. [...] And check conflicts in database ask [the agent] once*

Logic	SQLAlchemy only sees tables registered on the *same* `Base.metadata`. If the GPU tables lived in a separate models file with its own Base, Alembic autogenerate wouldn't see them next to `users`, foreign keys to `users.id` would fail to resolve, and we'd end up with two migration histories (two heads) that conflict on merge.
Rule I set	One `models.py` and one `main.py`; module code goes into the existing `logic/`, `routers/` and `schemas/` folders; one Alembic history.
Database conflict check	I asked for an explicit check before merging: table names, foreign-key targets and migration heads. His agent ran it and found no clash at the time, but flagged a likely future clash on a `labs` table.
Result (his log)	His code was moved into this layout with a relayout script, tested on throwaway copies of both branches, and `alembic check` showed an identical schema afterwards.

F2 How the GPU request tables should link

13 Oct, 12:11 (Adithya's log C9)

> **MY REVIEW QUESTION (relayed to the GPU owner)**
>
> *In labs table whats the primary key and foreign key, because when u create a gpu request and pull it up u need to create a table that stores this gpu with the id which ll be a foreign key from previous table [...]*

What I was checking	Every GPU request has to be tied to a user and to the GPU or lab it uses. I wanted the key structure confirmed before anything was merged into the shared schema.
Answer (his log)	A table-by-table key structure, with `jobs.user_id` referencing `users.id`.
Why this is right for my framework	My authorisation design deletes the `students` / `researchers` row when an admin revokes access (A2). If GPU jobs referenced `students.id`, a revoke would either be **blocked** (FK restrict: you can't delete a student with jobs) or **wipe the job history** (FK cascade). Pointing at `users.id`, which is never deleted by revoke, keeps the history and lets revoke work.
Decision	His tables already followed this chain, so I accepted the schema unchanged once that was confirmed (his C9).
Access rule it implies	Whether a user may submit a job is answered by whether an approved `students` / `researchers` row exists, not by the foreign key. I wrote this down in the framework README as the integration rule for every module.

Why the foreign key goes to users, not to the role table

```
users (id)  <-- never deleted by revoke
   |-- students / researchers (user_id)   <-- created on approval, DELETED on revoke
   |-- jobs (user_id -> users.id)         <-- job history must outlive a revoke
FK to students.id  + ON DELETE RESTRICT  -> admin revoke fails if the student has jobs
FK to students.id  + ON DELETE CASCADE   -> revoke silently deletes the job history
FK to users.id                           -> revoke works, history kept      (chosen)
can_submit(user) := exists(students/researchers row where user_id = user.id)
```

F3 Syncing before the GPU PRs were pushed

13 Oct, 12:18 (Adithya's log C10)

WHAT I FLAGGED

[I told him I had pushed changes to the backend] → recorded in his log as: 'let's pull workspace again, coz disha said she has made some changes'

Why it mattered	My framework and auth changes landed in the same files his PRs touched (`main.py`, `models.py`, the migrations). Pushing on top of an old `main` would have produced conflicts, or a second Alembic head.
Result (his log)	He fetched and compared with `origin/main` before pushing; nothing new had landed at that point, so no rebase was needed.

F4 Merging the GPU PRs into main

13 Oct, afternoon (Adithya's log C14)

WHAT I DID

[merged PRs #4, #5 and #6 on GitHub]

What was merged	#4 eligibility, priority classes and fair-share scoring; #5 the 10 GPU tables, partial unique indexes and the atomic claim; #6 POST /jobs and the scheduling pass. These were stacked branches (#5 on #4, #6 on #5), so they had to go in that order.
Review points that applied	The F1 layout (shared `models.py`, existing folders, one migration history) and the F2 key structure (`jobs.user_id` → `users.id`). Both were settled before these PRs went in.
Result (his log, C14)	Merged the same afternoon; `alembic current` matched the new head after syncing. The later PRs #7 and #9 went into the same structure.

F5 Shared docs: one API.md format

14 to 15 Oct (Adithya's log E4, F2)

WHAT I SET UP

[my API.md and README for the framework and auth were on main]

What happened (his log)	His `API_GPU.md` was written in the same format as my `API.md`, with cross-links into it, and the GPU sections of the README were drafted and handed to me because I own the README.
Why	One set of conventions (Bearer token, error format, status codes) defined once in the framework docs, and every module documented the same way.

Summary of design decisions

Area	Decision	Reasoning
Approval	Final status is derived from both decisions on every change	It can never be inconsistent with them
Approval	Reject is reversible; approval locks; revoke is separate and admin-only	Matches the portal; clean lifecycle; no orphaned rows
Concurrency	Row lock + UNIQUE constraints	Simultaneous approvals finalise exactly once
Accounts	Self-register only STUDENT/RESEARCHER; admin adds professors; script adds the first admin	No privilege escalation
Identity	Role re-read from the DB on every request	Role changes apply immediately; forged claims do nothing
Structure	routers → schemas → logic → models, central error handler	Thin routes; other modules plug in the same way
Database	PostgreSQL in Docker, not SQLite	Row locks actually work; same setup for the team
Schema	Alembic only; one DATABASE_URL; all timestamps UTC	No DuplicateTable; no drift; one time base
Passwords	Random professor password nobody sees; one OTP reset flow for everyone	Nothing to share, nothing to force
OTP	Hashed, 10 min, 5 tries, 1/min, same reply for every email	Brute force, spam and account discovery all blocked
Sessions	token_version in every JWT	A reset logs out every old session
Login	Email + password + role; one error message for any mistake	Role can't be discovered through the form
Integration	One models.py, one main.py, one Alembic history; modules use the existing folders	Every table on one metadata; no second migration head
Integration	Module tables reference users.id, not students/researchers	Revoke deletes role rows; job history must survive it
Integration	Access to modules = an approved role row exists	Authorisation stays in one place: the approval workflow
Integration	Sync with main before pushing; merge stacked PRs in order	Avoids conflicts in shared files and split heads